



DOCUMENTATION INTRODUCTION TO VR AND AR

WiSe 2025/26
ECATERINA COSCEEVA
(2222594)

IMS 3.Semester

Prof. Dr. Silvan Mertes

CONTENT

1	MINDLOG CONCEPT	S.3
	* ART BIBLE	S.5
2	MINDLOG ASSETS	S.6
	* 3D ENEMIES	S.6
	* OTHER 3D ASSETS	S.8
	* SOUNDS	S.9
3	UNITY	S.13
	* GAMELOOP	S.13
	* PLAYER	S.14
	* ENEMY	S.15
	* WEAPON	S.17
	* SOUND	S.19
4	* FUTURE WORK	S.20

MINDLOG CONCEPT



MindLog is a VR game that explores the psychological state of a player by placing imaginary monsters in one's real-time environment. It follows a vague plot where the player has been subjected to an increased amount of lucid dreaming over the span of a couple of weeks and now struggles to differentiate between fiction and reality. The game wants to challenge the feeling of uneasiness that comes with mixing the comfort and familiarity of one's surroundings with the unknown - mostly appearing in forms of different types of unidentified beings.

The following interaction concepts should be included in this game:

User-Object Interaction:

The player is able to interact with items like notes (reading through snippets of your own consciousness) while also being able to pick them up and hold onto them.

The creatures are able to hurt the player while the player is capable of causing them harm as well.

User-Object-Object Interaction:

The player is able to use items on creatures that appear within the game.

> *weapon system*

User Interface:

The User-Interface will let the player have access to a menu where they can pause or exit the game.

Additionally a health and sanity bar will be displayed to allow the player to keep track of their in-game condition.

Health Bar > will change depending on how many creatures have harmed the player and what type of items the player has consumed

Sanity Bar > will change depending on how actively the player will try to “clear” the level.

Taking care of the creatures and finding items > sanity high

Escaping the creatures and not actively picking up items > sanity low

Arms > The sanity level will additionally be viewed through the increasing amount of sparkles and patterns on the players arms

Audio:

Each one of the creatures will possess a unique sound to them, making the player become more aware of them being in close proximity prior to seeing them.

MixedRealityApp: using MetaQuest3 to move around within the game while simultaneously making use of the plane detection to interact with physical environment

MINDLOG ART BIBLE

To decide on what exactly should be used within the game, I tried to define my art bible first.

For the **art style** I wanted to go with something dreamy yet still unsettling. The horror genre usually uses dark settings and designs to awoke some type of fear within the players. In my case I tried to stray away from that original thought and lean into something more colorful. A prime example of a media that did this well was the movie *Midsommar* by Ari Aster. The use of bright colors and a seemingly “familiar place” might trick one first into thinking they’re in a safe environment. The person won’t suspect anything out of the ordinary to happen to them, just to be later completely taken away by the unexpected twist. And this was exactly what I tried to channel with MindLog.

The colors used would mostly be **saturated** (exception would be the design of the “Harpyie” figure) and **bright**. They would support the “**light and easy**” atmosphere at the beginning of the game.

This was something that would be applied to the 3D Models as well as the UI Design later on.



MINDLOG 3D ASSETS

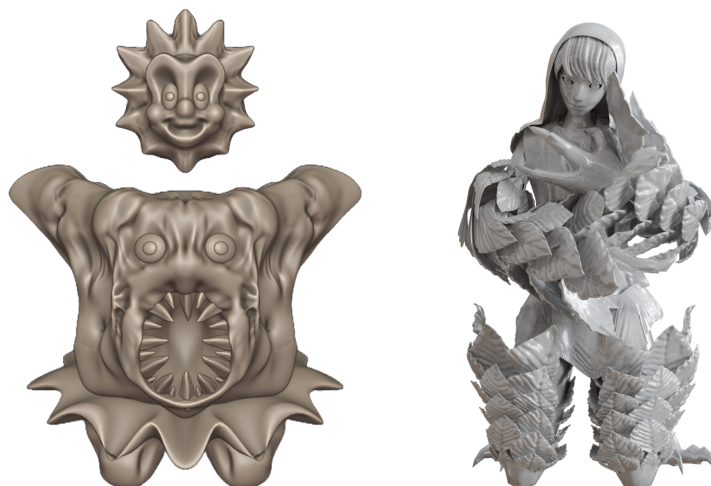
3D ENEMIES

After finalizing my thoughts on the visual guidelines I went directly into 3D modelling all the assets that I would need for this game.

The initial idea would be to create 3 different enemy models, a room, weapon assets, a health potion, interactable objects like a door, a toy or a ladder and once there was enough time, to create furniture assets as well.

All the 3D models were created with Blender, ideally with a hand-painted texture or temporarily with an already premade one.

For the enemy models I've started to first sculpt them, then retopologized them to a functional low-poly model (trying to lose as little details as possible in the process)



Once I've finished the enemy models, I created custom rigs for each of them and made 3 different types of base animations.

Later on when I configured the animator controller, it was important to me to include three states of animation.

The **basic idle** animation that would constantly play when the enemy is walking around and then tracking the player.

The **receiving damage** animation to give the player a visual feedback whenever the player successfully applied damage to the enemy

And most important - **the death animation** - to showcase the successful act of defeating your enemy.

Additionally an attacking animation would've been planned, it was then later on kind of mixed within the idle animation.

Even though funnily as of right now the "receiving damage" and "death" animations were scrapped in favor of the ragdoll system. For future work I would construct a standalone attacking animation to include in the animator controller of the enemy.



OTHER 3D ASSETS



The additional 3D assets I've created include the **weapons** as well parts of the **furniture**.

I experimented a bit and decided to make the weapons look like they're made out of **cardboard**.

Visually wise it would support the "dream state" concept of the game by making something harmless such as a cardboard material quite lethal when using it against the enemy.

It is possible to include even more weapon types - now I've chosen to create 3 of them, two melee and one ranged one. For the melee one I wanted to include something more ridiculous like a guitar, so I can later add a guitar sound to it whenever it collides with the monsters.

MINDLOG SOUNDS



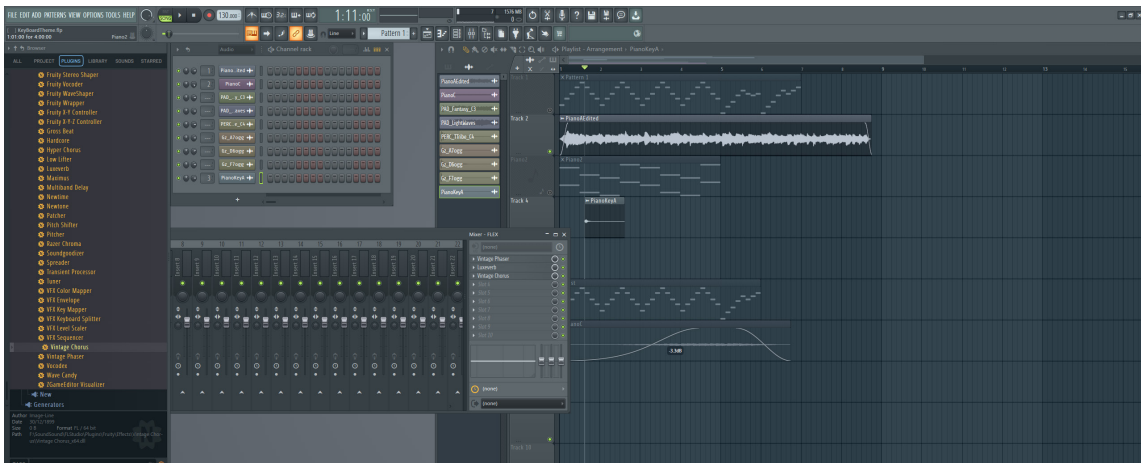
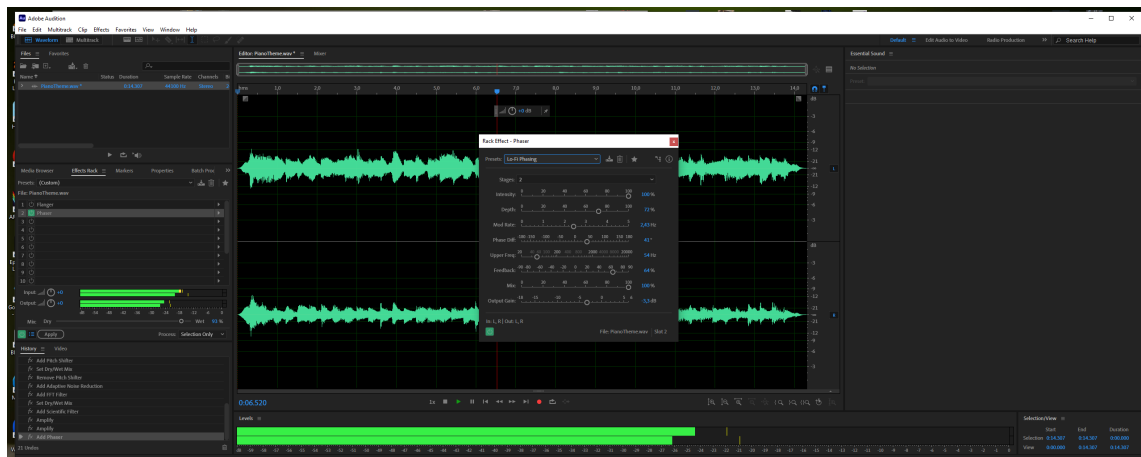
Sound List

The image above shows all the important sounds that will be created during for this game. The primary focus was on developing a theme that would be played in the intro right before the game as well as during the gameplay. This was to serve as a guideline for developing and editing the remaining sounds. Afterwards the focus shifted to the creature sounds and the player (combat) sounds. The UI sounds were to be edited last.

TOOLS + PROCESS

The **ZOOM H1essential** audio recorder was used to record the sounds.

The sounds were edited and finalized using **FL Studio** and **Adobe Audition**. Adobe Audition was mainly used to distort the sounds (custom presets were also created during the process), while all other steps, such as cutting and optimization, were carried out in FL Studio.



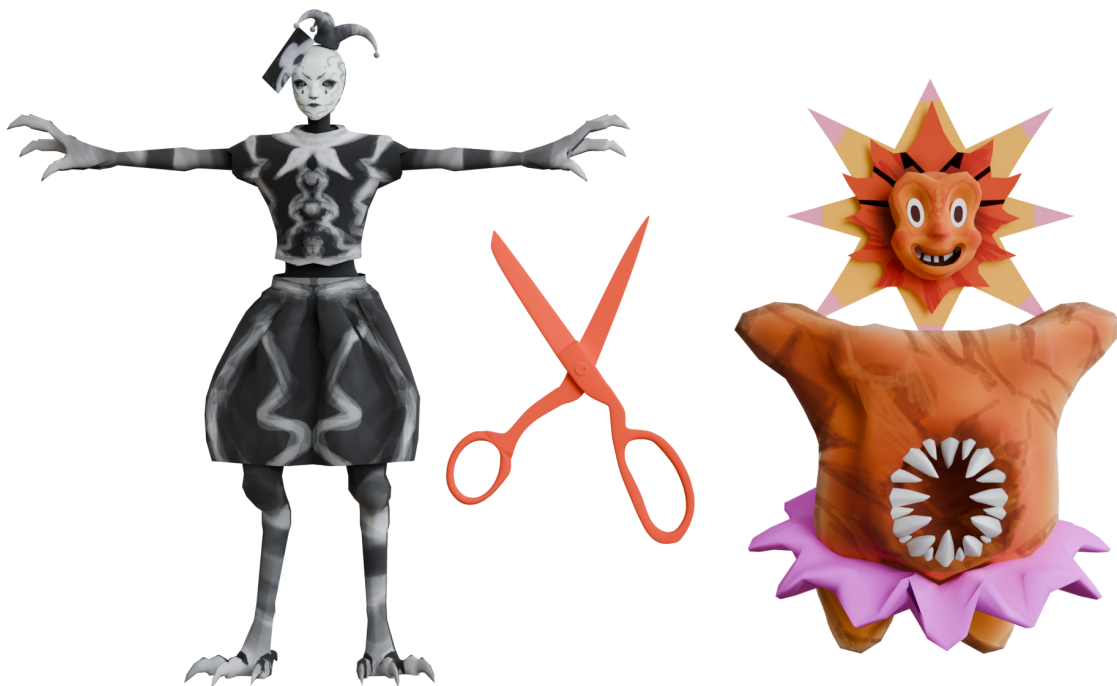
The first drafts of the ambient sounds were tested using a **piano**. UI sounds and the game theme were also created using piano sounds. These were organized and edited in **FL Studio**. Finally, the sound was converted using a “jukebox” plugin, and all of the original piano sounds now sound like the sounds of a jukebox. These are intended to reinforce the dreamlike and “childlike” aspect of the game.

The player sounds, especially the combat sounds, were created by recording a **knife** being deliberately stabbed into fruit or vegetables. With the help of the recorder, a very clean sound could also be recorded. Furthermore, a nutcracker, scissors, or a wooden spoon were used for additional attacks or “**hurt**” sounds. For the metallic sound, two metal objects were struck together.





For the **creature sounds**, I used the help of my two pets (dog and cat). I recorded and sorted all kinds of sounds, such as barking, meowing, panting, breathing, and paw/running noises on the floor. These were then imported into **Adobe Audition** and tested with various effects until the final versions for the different creatures were created.



UNITY

The engine I've used to implement all my assets as well as create the actual game was **Unity**.

Prior to that I already partially worked with it (for our master project). Thus while I've been a little familiar with the interface, my knowledge was only limited to importing assets.

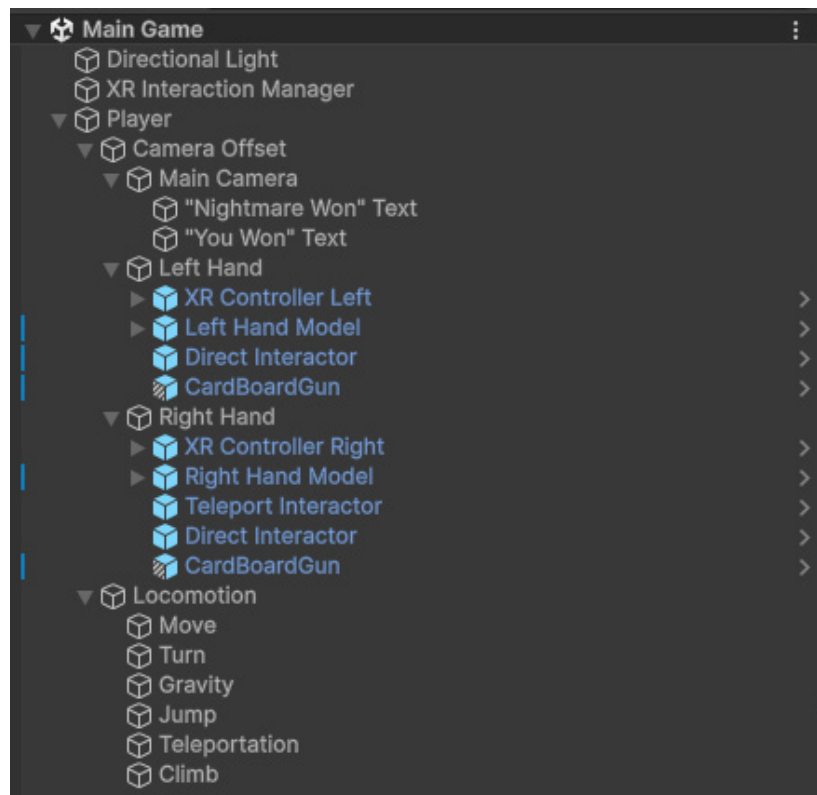
This is why the scripts that I've created for this game were mostly made by following tutorials such as those of "Valem Tutorials" on YouTube and the other scripts partially were based on posts that I've looked up on the unity discussion forums.

Some of those scripts were altered to either include some additional functions that I wanted to implement in the game or the other way around, I didn't want to have in.

GAMELOOP

The game loop of MindLog involves the player starting off in a room where they can grab a weapon and use them against a specific amount of monster that are being spawned in the level as a "nightmare that keeps following the player in the dream". Once a specific amount of enemies is killed, the player wins the game. And if the player dies, the level gets reset and you begin anew.

PLAYER



As I was setting up the Player / XR Origin I wanted to implement the Hand Models instead of Controllers to have a more immersive gameplay, especially when grabbing weapons. I attached the guns to the Players Hands as well. Both hands have the direct interactor prefab, including their colliders.

I played with the idea to include a full body rig, sadly there was not enough time to work on it, therefore it will be one of the future work aspects.

For the locomotion I wanted the player to have a continuous turn (right controller) and movement (left controller). It might cause a tad of motion sickness, hence why I included the teleportation as well, but for the gameplay it's better to have a more smoother movement.

Additionally the player is able to jump and climb on specific objects.

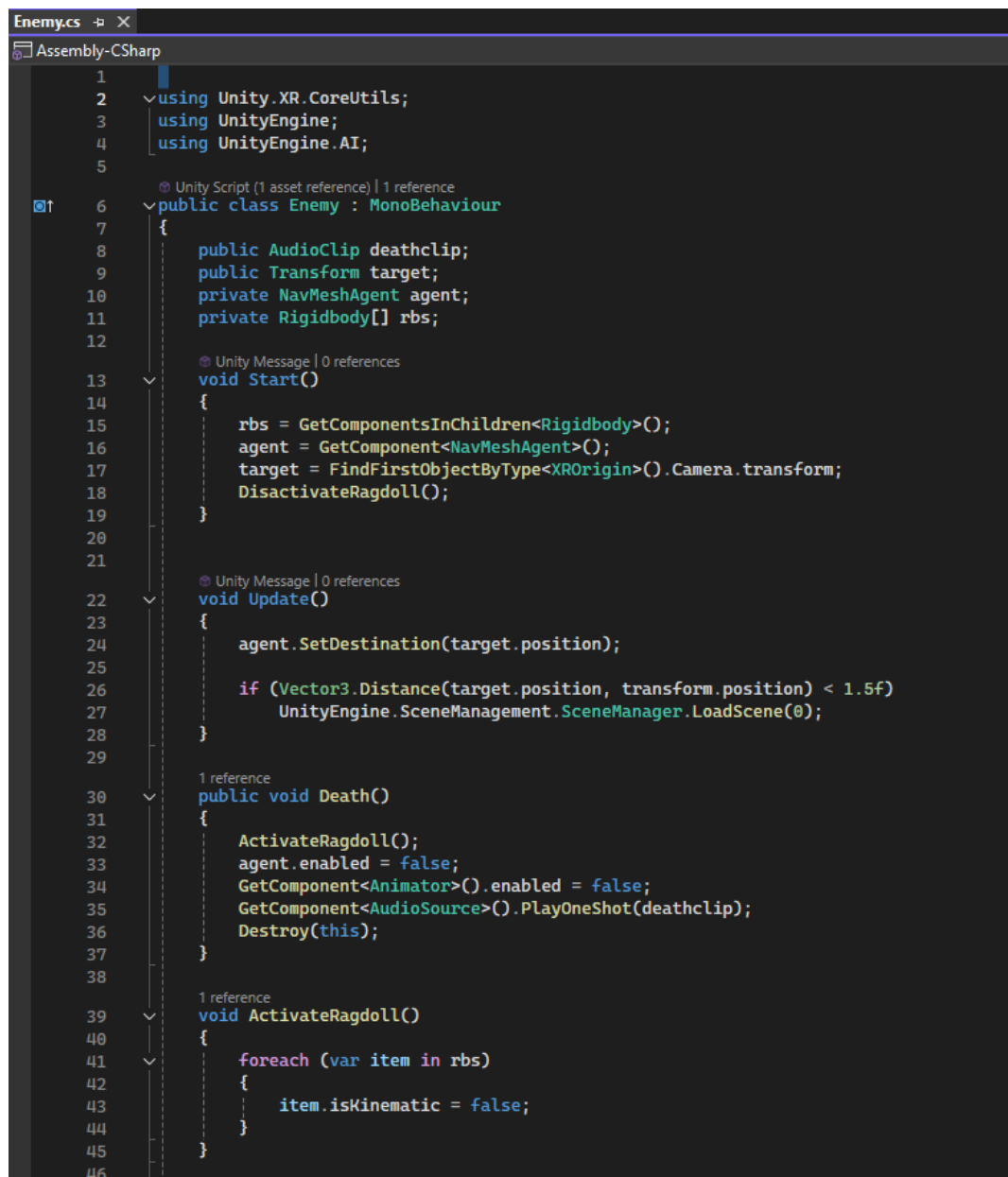
ENEMY

When constructing the enemy, I gave the model a Ragdoll, Rigidbody, NavmeshAgent, Animator Controller and Audio Source component.

The basic functions of the monster would be to follow the Player (XROrigin), and within the proximity of the player, the player would “die”.

Since I wanted the enemy to become a ragdoll yet still also play its animation, I deactivated the ragdoll when there is no damage done and once it does take the damage the ragdoll would activate and the animation would stop. Additionally I made a script for the different “limb” parts of the monster that would allow those parts to be “cut off” separately without damaging the whole monster. That is for all the limb parts except the “fatal” limb part, in this case I used the head, that would then kill the monster with a “one hit”. Once the monster dies it will play a separate audio clip indicating its end.

And so that there is a continuous spawning of enemies I also included a spawner that would then spawn the monsters from different set places within the level after a specific time frame.



```
1
2 using Unity.XR.CoreUtils;
3 using UnityEngine;
4 using UnityEngine.AI;
5
6 public class Enemy : MonoBehaviour
7 {
8     public AudioClip deathclip;
9     public Transform target;
10    private NavMeshAgent agent;
11    private Rigidbody[] rbs;
12
13    void Start()
14    {
15        rbs = GetComponentsInChildren<Rigidbody>();
16        agent = GetComponent<NavMeshAgent>();
17        target = FindFirstObjectByType<XROrigin>().Camera.transform;
18        DeactivateRagdoll();
19    }
20
21    void Update()
22    {
23        agent.SetDestination(target.position);
24
25        if (Vector3.Distance(target.position, transform.position) < 1.5f)
26            UnityEngine.SceneManagement.SceneManager.LoadScene(0);
27    }
28
29    public void Death()
30    {
31        ActivateRagdoll();
32        agent.enabled = false;
33        GetComponent<Animator>().enabled = false;
34        GetComponent<AudioSource>().PlayOneShot(deathclip);
35        Destroy(this);
36    }
37
38    void ActivateRagdoll()
39    {
40        foreach (var item in rbs)
41        {
42            item.isKinematic = false;
43        }
44    }
45
46
```

```
EnemySpawner.cs  x  Enemy.cs
Assembly-CSharp

1  using UnityEngine;
2
3  public class EnemySpawner : MonoBehaviour
4  {
5      public float spawnTime = 1;
6      public GameObject spawnGameObject;
7      public Transform[] spawnPoints;
8      private float timer;
9
10     void Start()
11     {
12     }
13
14
15
16     void Update()
17     {
18         if (timer > spawnTime)
19         {
20             Transform randomPoint = spawnPoints[Random.Range(0, spawnPoints.Length)];
21             Instantiate(spawnGameObject, randomPoint.position, randomPoint.rotation);
22             timer = 0;
23         }
24
25         timer += Time.deltaTime;
26     }
27
28 }
```

```
Limb.cs  x  GameManager.cs  EnemySpawner.cs  Enemy.cs
Assembly-CSharp

1  using UnityEngine;
2
3  public class Limb : MonoBehaviour
4  {
5      public bool fatal = false;
6      public void Hit()
7      {
8          // Limb childlimb = transform.GetChild(0).GetComponentInChildren<Limb>();
9          // if (childlimb)
10             // childlimb.Hit();
11             // This is for when I will later on create additional limb 3d models, so the ragdolls
12             // disappear on the monsters but will be visibly dropped to the ground
13
14             transform.localScale = Vector3.zero;
15
16             // GameObject spawnedLimb = Instantiate(limbPrefab, transform.parent);
17             // spawnedLimb.transform.parent = null;
18             // Destroy(spawnedLimb, 10);
19
20             if (fatal)
21                 GetComponentInParent<Enemy>().Death();
22
23             Destroy(this);
24         }
25
26     private void OnCollisionEnter(Collision collision)
27     {
28         if(collision.gameObject.CompareTag("Weapon"))
29             Hit();
30     }
31
32 }
```

WEAPON

For the gun I've decided to have it permanently attached to the players hands, making it usable at any point of time. It will also produce its own sound once the "bullet" takes off. To make it of immediate use I created another Action Map called "Weapon Toggler" and made the Button the Action Type. That Toggle Action is then connected to the Gun Activator Script. Upon pressing the button it will spawn the gun, and upon pressing it again, the gun will disappear and go back to the default hand model.

```
GunShooter.cs | SetTurnTypeFromPlayerPref.cs | SetTurnTypeFromPlayerPref.cs | PistolActivator.cs | GunShooter.cs
Assembly-CSharp
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.InputSystem; // Unity Input System
5
6  [Unity Script (3 asset references) | 0 references]
7  public class GunShooter : MonoBehaviour
8  {
9      public LayerMask layerMask;
10     [Tooltip("Assign an InputAction (e.g. a button) from your Input Actions asset.")]
11     public InputActionProperty shootingAction;
12
13     public LineRenderer linePrefab;
14     public GameObject rayImpactPrefab;
15     public Transform shootingPoint;
16     public float maxLineDistance = 5f;
17     public float lineShowTimer = 0.3f;
18     public AudioSource source;
19     public AudioClip shootingAudioClip;
20
21     [Unity Message | 0 references]
22     private void Update()
23     {
24         if(shootingAction.action.triggered)
25         {
26             Shoot();
27         }
28     }
29
30     [1 reference]
31     public void Shoot()
32     {
33         if (source != null && shootingAudioClip != null)
34         {
35             source.PlayOneShot(shootingAudioClip);
36         }
37
38         Ray ray = new Ray(shootingPoint.position, shootingPoint.forward);
39         bool hasHit = Physics.Raycast(ray, out RaycastHit hit, maxLineDistance, layerMask);
40
41         Vector3 endPoint;
```

```
42         if (hasHit)
43         {
44             endPoint = hit.point;
45         }
46         else
47         {
48             endPoint = shootingPoint.position + shootingPoint.forward * maxLineDistance;
49         }
50
51         if (linePrefab != null)
52         {
53             LineRenderer line = Instantiate(linePrefab);
54             line.positionCount = 2;
55             line.SetPosition(0, shootingPoint.position);
56             line.SetPosition(1, endPoint);
57             Destroy(line.gameObject, lineShowTimer);
58         }
59     }
60 }
61
62
63
64
65
```

```
GunActivator.cs  X GunShooter.cs
Assembly-CSharp
1  using UnityEngine;
2  using System.Collections.Generic;
3  using UnityEngine.InputSystem;
4
5  public class GunActivator : MonoBehaviour
6  {
7      public InputActionProperty toggleAction;
8      public bool activateOnStart = false;
9
10     public List<GameObject> goToActivateOnGunActivated;
11     public List<GameObject> goToActivateOnGunDeactivated;
12
13     private bool isActivated;
14
15
16     void Start()
17     {
18         isActivated = activateOnStart;
19         ApplyActivationState();
20     }
```



For the other Weapons I just gave them Colliders and Collisions and made them grabable for the Player.

SOUND

All sounds were implemented directly in **Unity**.

Since the current version of the game uses VR, it is not yet possible to interact with the actual environment.

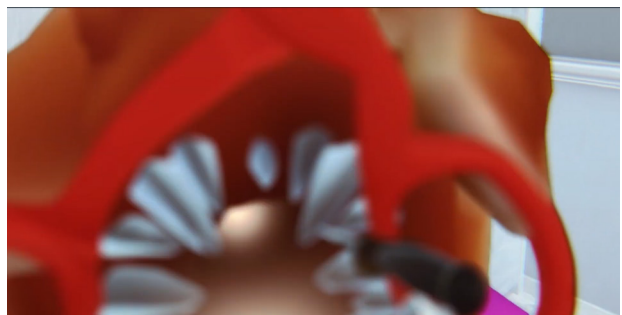
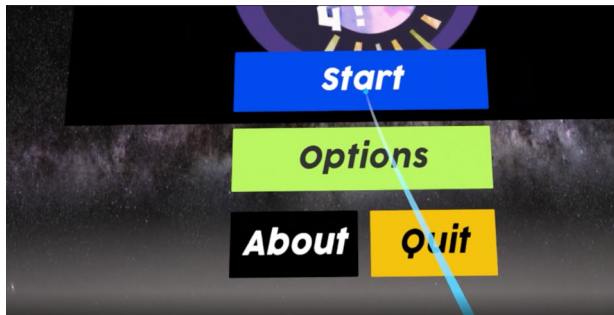
x Hovering over the buttons plays the UI sound.

x Both in the start menu and in the level, you can hear the MindLog theme playing in the background.

x You can already hear the monster in the level by its sound, and in VR it further enhances the immersion.

x The pick-up and drop sounds for the weapons have also already been implemented. You can hear these as soon as you pick up the knife or when it collides with the ground. The moment you stab the monster with the knife, you hear the “stab” sound.

x Other sounds such as “health drop” have not yet been implemented, but will be completed during the semester.



FUTURE WORK

Since I wasn't able to finish/complete the game loop, the first and foremost task for me would be to finish all the logic for a successful playthrough. Adding to that I would want to create a more complex health system, especially in relation to the player and include the concept of "healing" items that you can pick up inside the level.

Maybe also add additional interactable components that would make up for a fun game-play.

Therefore I'd probably create more 3D assets as well.

I'd love to create even more monsters for this and build up the lore that I thought of for this game. Most importantly fix the ragdoll / animator controller problem.

I want to include a nice looking and functioning UI in the future as well.

In relation to the health bar as well as the player menu within the game.

I really enjoyed making this game and experiencing making it on my own within unity for the first time as well. I would definitely want to finish it in the way that I imagined it and would love to put it on my portfolio page.